

STRUCTURA SI ORGANIZAREA CALCULATOARELOR

Calculatoare 3 2025-2026

CURS 11 Sapt 12 16 decembrie 2025 14:00-16:00

serban@upit.ro

Modul de notare:

Examen:	50%	
Proiect	20%	A3
Laborator:	20%	A2
Test (Midterm):	10%	A1

Obs. Fiecare componenta a notei finale (A1, A2, A3 si Examen) trebuie sa fie promovata cu nota minima 5.

Planificarea activitatilor:

Laborator:

- saptamanal, conform orarului;
- sapt. 13, 14 refaceri laboratoare;

Proiect:

- conform orarului

Curs:

- saptamanal, conform orarului;

Test (Midterm): in saptamana a 10-a (9 decembrie 2025)

Continut

Bazele aritmetice ale calculatoarelor electronice

- Sisteme de numeratie

- Reprezentarea numerelor in virgula fixa / mobila

- Operatii aritmetice in virgula fixa / mobila

Proiectarea blocurilor logice pentru operatiile aritmetice in calculatoarele electronice

- Adunarea; scaderea

- Inmultirea

- Impartirea

Proiectarea procesoarelor

- Procesoare RISC

- Setul de instructiuni

- Calea de date (Data Path)

- Calea de control (Control Path)

Bibliografie

David A. Patterson, John L. Hennessy - Computer Organization and Design The Hardware Software Interface [RISC-V Edition] (6th edition Morgan Kaufmann, 2018)

David Harris, Sarah Harris - Digital Design and Computer Architecture (2007)

J. Hennessy, D. Patterson - Computer Architecture. A Quantitative Approach (6th edition Morgan Kaufmann 2019)

William Stallings - Computer Organization and Architecture Designing for Performance (10th edition Pearson 2016)

A. Tanenbaum, T. Austin - Structured Computer Organization, (6th edition Pearson 2013)

Karbo M.B. - A complete illustrated guide to the PC hardware (2005)

ALGORITMUL BOOTH PENTRU EFECTUAREA OPERAȚIILOR DE ÎNMULȚIRE A NUMERELOR BINARE ÎNTREGI

Pentru exemplul prezentat constatăm faptul că numărul de produse parțiale ce urmează să fie cumulate pentru obținerea produsului final este egal cu 4. Ne propunem să micșorăm acest număr de produse parțiale ce intra în calculul produsului final.

Considerăm următoarea operație de înmulțire: $101011_{(2)} \cdot 011110_{(2)}$

Pentru înmulțitor avem: $011110_{(2)} = 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 = 30_{(10)}$

Cunoaștem ca: $a^n - b^n = (a - b)(a^{n-1} + a^{n-2} \cdot b + \dots + a \cdot b^{n-2} + b^{n-1})$ Cu $a = 2, b = 1$ obținem

$$2^n - 1 = 2^{n-1} + 2^{n-2} + \dots + 2 + 1;$$

$$2^n - 2 = 2^{n-1} + 2^{n-2} + \dots + 2$$

Rezulta: $2^4 + 2^3 + 2^2 + 2^1 = 1 \cdot 2^5 - 1 \cdot 2^1 = 1 \cdot 2^5 + (-1) \cdot 2^1$ de unde: $011110_{(2)} = 100000_{(2)} + (-10_{(2)})$

Înmulțirea dată devine:

$$101011_{(2)} \cdot (100000_{(2)} - 10_{(2)}) = 10101100000_{(2)} - 1010110_{(2)} = 10101100000_{(2)} + (-00001010110_{(2)}) =$$

$$= 10101100000_{(2)} + 11110101010_{(2)} = 10100001010_{(2)}$$

$$(-00001010110_{(2)}) = 11110101001 + 1 = 11110101010_{(2)}$$

La ultima operație de adunare a doi operanți în CC2 bitul de depășire este egal cu 0 și rezultatul obținut pe 11 biți este cel corect, neglijându-se bitul de transport apărut. Reprezentarea în CC2 pentru operația menționată s-a făcut pe 11 biți.

Algoritmul lui Booth propune formarea unui șir de coeficienți prin analiza biților înmulțitorului.

Termenii care apar în acest șir sunt -1, 0 sau 1, indicând și operația care se efectuează.

Șirul menționat se formează astfel:

- Înmulțitorul se analizează de la dreapta la stânga;
- Pentru primul bit 1 obținut în șirul lui Booth se alocă un coeficient -1 (produsul parțial se scade – folosind CC2);
- Dacă în continuare se regăsește un bit 1 coeficientul din șirul lui Booth este 0 (produsul parțial se deplasează stanga);
- Dacă următorul bit întâlnit este 0 coeficientul din șirul lui Booth este 1 (produsul parțial se adună);

Folosind șirul coeficienților Booth orice înmulțitor (număr binar) se poate scrie ca sume, respectiv diferențe de numere binare care în zecimal sunt puteri întregi ale lui 2.

$$\begin{array}{r} 101011 \times \\ 011110 \\ \hline 101011 \\ 101011 \\ 101011 \\ 101011 \\ \hline 1010001010 = 50A_h \end{array}$$

Folosind șirul coeficienților Booth, orice înmulțitor (număr binar) se poate scrie ca sume, respectiv diferențe de numere binare care în zecimal sunt puteri întregi ale lui 2.

Ex.1 Pentru $0101111_{(2)}$ sirul Booth este $1 -1 \ 1 \ 0 \ 0 \ 0 \ -1$ rezultand $101111_{(2)} = 1000000_{(2)} - 100000_{(2)} + 10000_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 4 produse partiale, fata de 5 produse partiale in algoritmul clasic.

Ex.2 Pentru $011011_{(2)}$ sirul Booth este $1 \ 0 \ -1 \ 1 \ 0 \ -1$ $11011_{(2)} = 100000_{(2)} - 1000_{(2)} + 100_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 4 produse partiale, similar cu 4 produse partiale care apar in algoritmul clasic.

Ex.3 Pentru $010101_{(2)}$ sirul Booth este $1 \ -1 \ 1 \ -1 \ 1 \ -1$ $10101_{(2)} = 100000_{(2)} - 10000_{(2)} + 1000_{(2)} - 100_{(2)} + 10_{(2)} - 1$

Se constata ca prin algoritmul Booth obtinem 6 produse partiale, fata de 3 produse partiale care apar in algoritmul clasic.

În concluzie, această primă formă a algoritmului lui Booth este eficientă atunci când șirul de biți 1 logic aflați în înmulțitor este mai mare ca 2.

Exemplu $21 \times 14 = 294$

0 1 0 1 0 1	×	Deinm
0 0 1 1 1 0		Inm

0 +1 0 0 -1 0		Sir Booth Inm

(- 0101010)	+	Prod.part.1
(0101010000)		Prod.part.2

0 1111010001		Transport
1111010101	+	CC2 PP1
1		
0101010000		PP2

0100100110		Produs

0 = C _{ET9} = 1 ⊕ 1 = 0		

ALGORITMUL BOOTH MODIFICAT

Acest algoritm oferă soluții pentru situația în care șirul de biți 1 logic aflați în înmulțitor nu este mare, inclusiv pentru situația biților 1 solitari. În acest algoritm modificat se propune un nou șir de coeficienți derivați din algoritmul lui Booth normal. Aceștia apar din analiza fiecărui grup de câte 2 termeni din șirul Booth normal, plecând din dreapta către stânga. În noul șir termenii care pot să apară sunt ± 2 , ± 1 și 0.

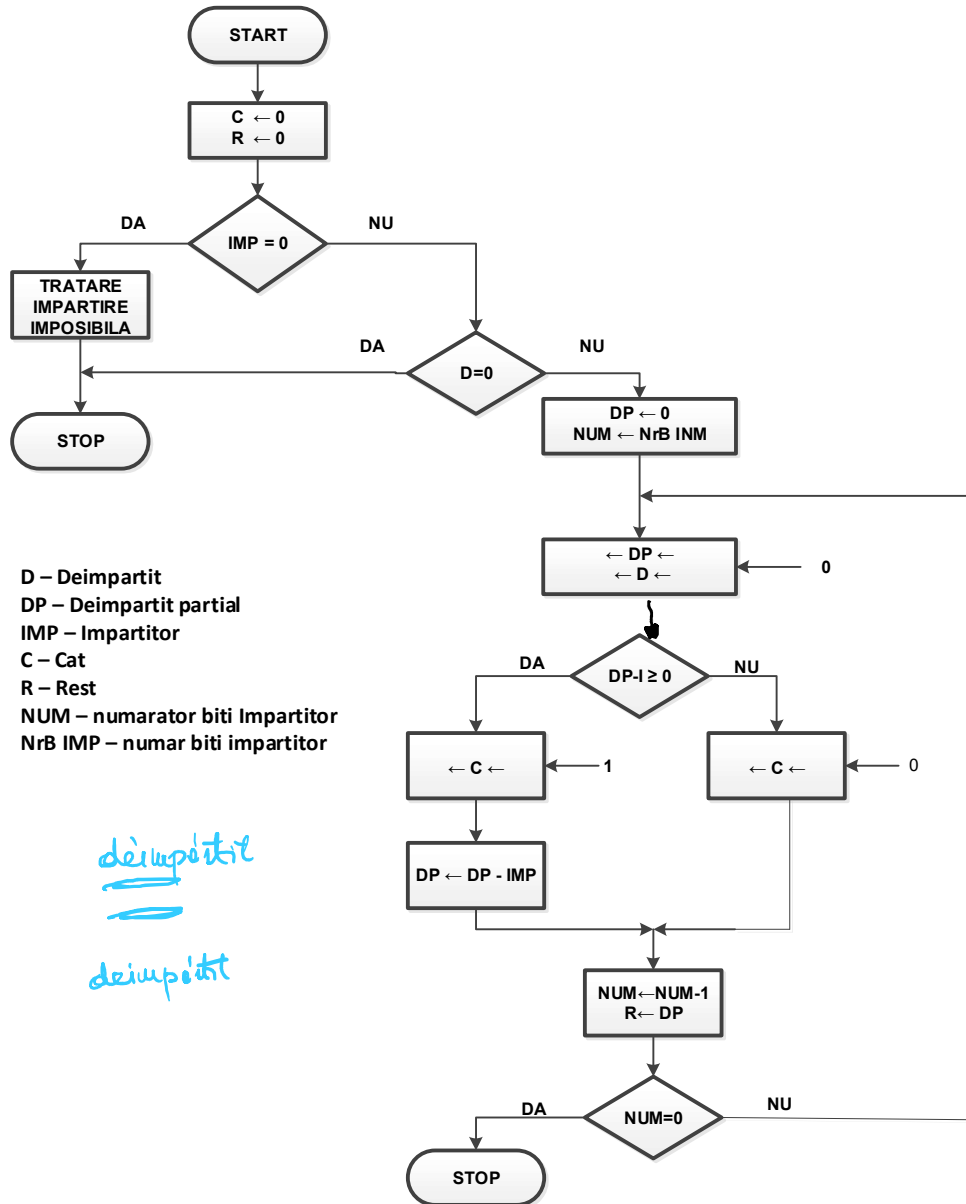
Sucesiune biți în nr binar	Coeficienți în Booth normal	Coeficient în Booth modificat
0 0 0	0 0	0
1 1 1		
0 1 1	1 0	+2
0 0 1	0 1	+1
1 0 0	-1 0	-2
0 1 0	1 -1	+1
1 1 0	0 -1	-1
1 0 1	1 -1	+1

Exemplu $14 \times 21 = 294$

0 0 1 1 1 0	×	Deinm
0 1 0 1 0 1		Inm
1 -1 1 -1 1 -1		Sir Booth Inm
↓ ↓ ↓		
+1 +1 +1		Sir Booth mod.
2^4 2^3 2^0		Ponderi coef.
0 0 1 1 1 0		PP1
0 0 1 1 1 0 0 0		PP2
0 0 1 1 1 0 0 0 0 0		PP3
0 0 0 1 1 1 1 0 0 0		Transport
0 1 0 0 1 0 0 1 1 0		Produs

END 02 dec 2025

CIRCUIT HARDWARE PENTRU EFECTUAREA OPERATIEI DE IMPARTIRE



deimpartit
deimpartit

DP	D
----	---

 $\leftarrow 0$
 $I \equiv IMP$

123456 | 89
 89 ↓
 344
 267
 = 775
 712
 = 746
 712
 34
 CAT
 REST

10110101 | 101
 101 ↓ ↓ ↓
 000101
 101 ↓
 00001
 CAT
 REST

DP	D
----	---

IMP

NUM DEIMP (biti)
 ↓

NUM

 (NrB)

Proiectarea unui MPU de tip RISC

RISC – Reduced Instruction Set Computer

CISC - Complex Instruction Set Computer

Un program este o secventa de instructiuni care permite procesarea de informatii de catre un procesor.

Efectuarea unei instructiuni se poate realiza in 2 moduri:

- prin interpretare, iar in acest sens instructiunea se descompune in microinstructiuni;
- prin executare, instructiunea fiind realizata direct de structura hardware (cablat), fara vreo intermediere.

Procesor interpretor – opereaza folosind logica microprogramata, fiecare instructiune fiind executata intr-un automat microprogramat prin intermediul mai multor microinstructiuni ce alcatuiesc microprogram.

Procesor executiv – opereaza folosind logica cablata, fiecare instructiune fiind executata direct de hardware, fara folosirea de automate microprogramate.

Un MPU RISC este un procesor executiv care executa direct instructiunile primite, fara a le interpreta folosindu-se de structura sa interna si fara a folosi automate microprogramate.

Proiectarea unui MPU RISC presupune parcurgerea urmatoarelor activitati:

- Proiectarea setului de instructiuni (instruction set architecture);
- Proiectarea caii de date (data path), adica a structurilor digitale si respectiv a magistralelor de comunicatii dintre acestea care permit executia propriu-zisa a instructiunilor;
- Proiectarea caii de control (control path), adica a structurii logice care gestioneaza circuitele digitale ce formeaza calea de date, oferind comanda acestor circuite in vederea executiei instructiunilor.

Arhitectura unui MPU RISC este dependenta de setul de instructiuni pe care il executa. O modificare de instructiune poate atrage schimbari majore la nivelul structurii hardware a procesorului.

Caracteristicile procesorului de tip RISC care va fi proiectat

Procesorul va avea o arhitectura Harvard cu spatii de memorie distincte pentru programe, respectiv date.

Organizarea memoriei de programe va fi de 64k x 32, iar cea a memoriei de date 64k x16.

Procesorul va dispune de 2 magistrale distincte de adrese de cate 16 biti, una fiind folosita pentru adresarea memoriei de programe, cealalta pentru adresarea memoriei de date. Va mai dispune si de 2 magistrale de date, una pe 32 de biti cu are sens de intrare in procesor, fiind folosita pentru preluarea codurilor instructiunilor (citire de catre UC), cealalta pe 16 biti, care permite tranferul de informatii intre procesor si memoria de date, aceasta fiind bidirectionala si permitand operatii de citire si scriere. Informatiile transferate pe aceasta magistrala sunt operanzi ce urmeaza a fi procesati in interiorul procesorului, in situatia citirii acestora, respectiv rezultate depuse de procesor in memoria de date pentru situatia scrierii acestora.

Procesorul va fi actionat de un semnal de tip ceas digital, fiecare instructiune urmand a fi executata intr o singura perioada a acestui semnal de ceas.

Procesorul va putea sa execute un total de 32 de instructiuni, fiecare fiind reprezentata pe un dublu cuvânt (32 biti).

Procesorul va dispune de un set de 32 de registri interni, fiecare cu dimensiunea de 16 biti, care se vor comporta similar pentru instructiunile ce formeaza setul de instructiuni (nu va exista vreun registru privilegiat). Aceasta proprietate poarta denumirea de ortogonalitatea setului de instructiuni in raport cu registrii procesorului.

Pentru simplificarea arhitecturii hardware, mecanismul stivei va fi redus la minim. In acest sens, 1 din cei 32 de registri din setul de registri, va fi utilizat pe post de stiva. Acest registru va contine varful stivei, de unde si denumirea acestuia TOS – top of stack. Registrul TOS este cel al 32-lea din setul de registri. Aceasta simplificare conduce la o stiva formata dintr-o singura locatie, fiind astfel posibila la procesorul proiectat fie apelarea unei singure subrutine, fie tratarea unei singure intreruperi, nefiind posibil apel imbricat de subrutine sau depuneri multiple in stiva.

Proiectarea unui MPU de tip RISC – cont.

1. Proiectarea setului de instructiuni (instruction set architecture)

Conform caracteristicilor enuntate, procesorul RISC propus va avea instructiunile reprezentate pe 32 de biti. Instructiunile din setul propus, vor avea urmatoarele formate binare posibile:

Formatul 1:

31	-	27	26	-	22	21	-	17	16	-	12	11	-	0
COD			d			l			r			-		

unde:

COD = codul masina al instructiunii;

d = destinatie rezultat (registru in setul de registri);

l = operand stang (registru in setul de registri);

r = operand drept (registru in setul de registri).

Formatul 1 va fi folosit pentru instructiuni de tipul:

OP d, l, r (d ← l Operatie r) depune in destinatia **d** rezultatul operatiei **OP** dintre operandii **l** si **r**.

Formatul 2:

31	-	27	26	-	22	21	-	17	16	15	-	0
COD			d			l			-	v		

unde:

COD = codul masina al instructiunii;

d = destinatie rezultat (registru in setul de registri);

l = operand stang (registru in setul de registri);

v = valoare imediata (operand imediat).

Formatul 2 va fi folosit pentru instructiuni de tipul:

OP d, l, v (d ← l Operatie v) depune in destinatia **d** rezultatul operatiei **OP** dintre operandii **l** si **v**.

Din formatele reprezentate => procesorul va efectua 2^5 instructiuni deoarece COD este format din 5 biti.

Mai rezulta ca d poate avea $2^5 = 32$ valori. Operandul l poate avea 32 de valori ca si operandul r.

Valorile care le poate lua d reprezinta adrese destinatie, de fapt vor fi registri aflati intr-un set de registri pe care procesorul il va avea in dotare. In mod similar l si r reprezinta adrese ale operandilor respectivi, de fapt registri aflati in acelasi set de registri => procesorul pe care il vom proiecta va avea un set de 32 de registri care pot fi si surse ale operandilor, dar si destinatii pentru rezultat.

Pentru ca acest lucru sa fie posibil, setul de registri va fi dotat cu un mecanism multiplu de adresare care sa permita simultan adresarea a 2 registri pe post de operand respectiv al unui registru destinatie pentru plasarea rezultatului instructiunii.

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

Instructiunile procesorului propus sunt grupate in urmatoarele categorii:

- a) Instructiuni functionale – categorie ce cuprinde instructiuni aritmetice, logice si alte instructiuni.
- b) Instructiuni de salt – cuprinde instructiuni de salt conditionat si neconditionat.
- c) Instructiuni de control

a) Instructiuni functionale:

a1). Instructiuni aritmetice:

- 1. ADD d,l,r <==> $d \leftarrow l + r$; F1
- 2. SUB d,l,r <==> $d \leftarrow l - r$; F1
- 3. ADDV d,l,v <==> $d \leftarrow l + v$; F2
- 4. SUBV d,l,v <==> $d \leftarrow l - v$; F2

a2). Instructiuni logice:

- 5. AND d,l,r <==> $d \leftarrow l \wedge r$; F1
- 6. OR d,l,r <==> $d \leftarrow l \vee r$; F1
- 7. XOR d,l,r <==> $d \leftarrow l \oplus r$; F1
- 8. NOT d,l <==> $d \leftarrow \bar{l}$; F1, F2

a3). Alte instructiuni:

- 9. SHR d,l <==> $d \leftarrow l/2$; F1, F2
- 10. SHL d,l <==> $d \leftarrow 2 * l$; F1, F2
- 11. MOV d,l <==> $d \leftarrow l$; F1, F2 (move – transfera continutul unui registru sursa intr-un registru destinatie)
- 12. LOAD d,l <==> $d \leftarrow (l)$; F1, F2 (permite citirea unei locatii de memorie externa de date a carei adresa este precizata in registrul l)
- 13. STORE l, r <==> $(l) \leftarrow r$; F1 (permite scrierea unei locatii de memorie externa de date a carei adresa este precizata in registrul l)
- 14. VAL d, v <==> $d \leftarrow v$; F2
- 15. CR d, l, r <==> $d \leftarrow Cy(l + r)$ (permite adunari cu operanzi exprimati pe mai multe cuvinte)
- 16. BR d, l, r <==> $d \leftarrow Bw(l - r)$ (permite scaderi cu operanzi exprimati pe mai multe cuvinte)

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

b) Instructiuni de salt

b1). Salturi neconditionate:

17. JMP v $\Leftarrow$ PC $\leftarrow$ v ; F2 (salt absolut)
18. JMR v $\Leftarrow$ PC $\leftarrow$ PC + 1 + v ; F2 (salt relativ)

Aceasta instructiune incarca PC cu o adresa a carei valoare depinde valoarea anterioara. Continutul lui v poate fi in acest caz un numar exprimat in CC2 pe 16 biti care va indica distanta de salt la care va ajunge procesorul in memoria de program, luand ca referinta adresa existenta in PC (PC +1).

b2). Lucru cu subrutine:

19. CALL v $\Leftarrow$ TOS $\leftarrow$ PC + 1, PC $\leftarrow$ v; TOS= top of stack (subrutina cu apelare absoluta)
Apeleaza o subrutina aflata la adresa absoluta v in memoria de programe. Pentru acest lucru salveaza continutul PC-ului, adica adresa de la care se face reluarea programului dupa revenirea din subrutina, in stiva. Pentru simplificarea procesorului proiectat am considerat ca in locul stivei este folosit un unic registru din setul de registri pe care notat TOS.
20. CALR v $\Leftarrow$ TOS $\leftarrow$ PC + 1, PC $\leftarrow$ PC + 1 + v subrutina cu apelare relativa - Call relative); v este reprezentat in CC2
21. RET $\Leftarrow$ PC $\leftarrow$ TOS (revenire din subrutina - Return)

b3). Salturi conditionate:

22. CJZ I, v $\Leftarrow$ PC $\leftarrow$ v; I != 0 $\Rightarrow$ PC $\leftarrow$ PC + 1 F2 (compare and jump if zero)
23. CJNZ I, v $\Leftarrow$ PC $\leftarrow$ PC + 1 ; I != 0 $\Rightarrow$ PC $\leftarrow$ v F2 (compare and jump if not zero)
24. CJRZ I, v $\Leftarrow$ PC $\leftarrow$ PC + 1 + v ; I != 0 $\Rightarrow$ PC $\leftarrow$ PC + 1 F2 (compare and jump relative if zero); v este CC2
25. CJRNZ I, v $\Leftarrow$ PC $\leftarrow$ PC + 1 ; I != 0 $\Rightarrow$ PC $\leftarrow$ PC + 1 + v F2 (compare and jump relative if not zero) ; v este CC2

OBSERVATIE: Instructiunile cu adresare relativa permit relocarea programelor in memoria de programe a sistemului de calcul, adica mutarea acestor programe in diverse zone de memorie, pastrandu-se proprietatea de a rula in mod similar cu aceleasi rezultate, indiferent de zona de memorie in care sunt mutate. Folosind relocabilitatea se poate defragmenta memoria atunci cand este folosita pentru mai multe programe.

1. Proiectarea setului de instructiuni (instruction set architecture) – cont.

c). Instructiuni de control

- 26. HALT – oprire procesor
- 27. NOP – no operation
- 28. DI – Disable interrupt
- 29. EI – Enable interrupt

Ultimele 2 instructiuni vor permite sa implementam un mecanism de intreruperi mascabil la nivelul procesorului. Procesorul va dispune de o intrare pentru primirea cererilor hardware de intreruperi care vor putea fi validate sau invalidate pe cale software, folosind instructiunile mentionate.

La instructiunile prezentate, care se vor putea prelua din memoria de programe, se adauga inca 2 instructiuni (comenzi) declansate la nivelul procesorului ca urmare a aplicarii unor semnale externe:

- 30. RESET <==> PC = 0 (initializarea procesorului); se declanseaza ca urmare a aparitiei unui semnal de tip RESET
- 31. INT <==> TOS $\leftarrow$ PC+1 ; PC $\leftarrow$ AI (adresa subrutina tratare intreruperi) se declanseaza la primirea unei intreruperi externe (dupa rulara EI)

16 dec 2025